Revolving NBP-Like Decoders for Cyclic and Quasi-Cyclic LDPC Codes

Qinshan Zhang, Bin Chen, *Member, IEEE*, Tianqu Zhuang, Yong Jiang, *Member, IEEE*, and Shu-Tao Xia, *Member, IEEE*

***Abstract*—Recently, deep learning has demonstrated improvements over classical decoding algorithms in various families of error-correcting codes with short to moderate block lengths. Neural decoders like neural belief propagation (NBP), constructed based on the Tanner graphs of linear codes, have been extensively studied, but their application to longer codes is limited due to increased network complexity with longer block lengths. This complexity leads to higher computational costs and deployment challenges, such as GPU memory usage. To address this, we propose a novel revolving framework for NBP-like decoders tailored to cyclic and QC-LDPC codes, a commonly used class of error correction codes. Our approach leverages the cyclic structure and section-wise cyclic structure inherent in cyclic and QC-LDPC codes respectively, significantly simplifying the complexity of the network. Experimental results demonstrate the effectiveness of our method across various cyclic and QC-LDPC codes. Especially, compared to the traditional decoding scheme based on the section-wise cyclic structure, our proposed decoder shows substantial improvements on 5G LDPC codes, exhibits better performance than the traditional min-sum decoder, and approaches the sum-product decoder without a noticeable error floor within the investigated noise levels.**

***Index Terms*—LDPC decoder, neural decoder, neural belief propagation, QC-LDPC, cyclic LDPC, parity-check matrix.**

I. Introduction

LOW-DENSITY parity-check (LDPC) codes, as an important class of error-correcting codes, show many elegant properties and excellent performance, and have received great attention since it was rediscovered in the late 1990s. LDPC codes generated randomly usually suffer the drawback that their high complexity of encoding and decoding is unacceptable for practical applications. Therefore, people prefer structured LDPC codes constructed with different useful tools, including algebraic and graph approaches [1], [2], [3]. These LDPC codes are mainly characterized by cyclic or quasi-cyclic (QC) parity-check matrices. Furthermore, cyclic and QC structures inherent in these codes make it possible to utilize the properties to improve the throughput of hardware implementations for decoding, with fewer wires and processing units [2], [4], [5], [6]. In general, an LDPC code can be represented by the Tanner graph based on its parity-check matrix, which consists of two separate parts of nodes termed variable nodes (VNs) and check nodes (CNs), and there only exist edges that connect two nodes from different parts. It is widely known that the belief propagation (BP) algorithm performs iteratively to decode LDPC codes. Specifically, the sum-product algorithm (SPA) shows the best performance in most situations, while its high complexity limits its practicality. The min-sum algorithm (MSA) and its variants, such as the normalized min-sum algorithm (NMSA) and offset min-sum algorithm (OMSA), adopt the approximations of messages, accelerating the overall decoding speed at the cost of acceptable performance degradation in practice.

By integrating the structural characteristics of LDPC codes with the BP algorithm, a new decoding method can be obtained, employing specially simplified parity-check matrices to perform message updating. In addition, processing units of the decoder can be reused through cyclic shifting [4] or *section-wise* cyclic shifting [5]. These approaches, referred to as revolving iterative decoding (RID), lead to a noteworthy reduction in the number of hardware circuits and processing units in the decoder implementation, achieving a significantly streamlined decoder with very small performance degradation compared to NMSA. Although its methodology is simple and intuitive, the RID scheme reduces the complexity of decoders for algebraic LDPC codes, with negligible performance degradation. However, this decoding algorithm is originated initially from the goal of optimizing decoding for algebraic cyclic and QC-LDPC codes, and its performance on LDPC codes obtained through other construction methods has not been directly and thoroughly validated. In recent years, deep-learning-driven decoder design has attracted much attention due to its promising potential. Deep neural networks (DNNs) can automatically learn parameters in an end-to-end fashion. Leveraging this characteristic, Nachmani et al. [7] proposed the neural BP (NBP) network. NBP is based on the Tanner graph and constructs an unfolded feed-forward trellis graph. Throughout the message-passing process (commonly referred to as forward propagation in deep learning), NBP assigns learnable weights to the messages transmitted along each edge,

Received 7 February 2025; revised 7 July 2025 and 28 August 2025; accepted 13 October 2025. Date of publication 22 October 2025; date of current version 2 January 2026. This work is supported in part by the National Natural Science Foundation of China under grant 62571298, 62576122, 62301189, and Shenzhen Science and Technology Program under Grant KJZD20240903103702004, JCYJ20220818101012025, GXWD20220811172936001. The associate editor coordinating the review of this article and approving it for publication was M. Shirvanimoghaddam. *(Corresponding author: Bin Chen.)*

Qinshan Zhang, Yong Jiang, and Shu-Tao Xia are with Tsinghua Shenzhen International Graduate School, Shenzhen 518055, China, and also with the Peng Cheng Laboratory, Shenzhen 518066, China (e-mail: zhangqs24@mails.tsinghua.edu.cn; jiangy@sz.tsinghua.edu.cn; xiast@sz.tsinghua.edu.cn).

Bin Chen is with Harbin Institute of Technology (Shenzhen), University Town of Shenzhen, Nanshan, Shenzhen 518055, China (e-mail: chenbin2021@hit.edu.cn).

Tianqu Zhuang is with Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen 518055, China (e-mail: zhuangtq23@mails.tsinghua.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TCOMM.2025.3624159, provided by the authors.

Digital Object Identifier 10.1109/TCOMM.2025.3624159

thereby adjusting the importance of different messages and mitigating the negative impact of short cycles in the Tanner graph on the iterative decoding performance, especially when decoding high-density parity-check (HDPC) codes as is shown in [8].

Similar to the optimization of standard SPA, variants of NBP based on NMSA and OMSA were proposed, namely neural NMS (NNMS) network [8] and neural OMS (NOMS) network [9]. These adaptations, while sacrificing a little performance, yield networks with lower computational complexity. This parallels the optimization strategy seen in the traditional BP algorithm, while another line of research aims to refine NBP networks from the perspective of deep learning. Buchberger et al. reduced the number of hidden layer nodes in the NBP network by pruning some nodes identified as relatively less important [10]. In the context of applying NBP to cyclic codes, Chen and Ye exploit the cyclical shift property to share weights among messages that satisfy the cyclical shift relationship [11]. Dai et al. shifted their focus on decoding protograph LDPC codes, which are derived from base matrices. They implement weight sharing among edges copied from the same edge in the protograph [12]. Nachmani and Be'ery employed a deeper NBP as the teacher and a shallower NBP as the student, attempting to make the performance of the student NBP approximates that of the teacher NBP by knowledge distillation [13]. Shah and Vasavada emulated the layer decoding strategy to construct the NNMS network, where each hidden layer indexed by odd numbers updates a subset of CN messages, enabling NBP to decode in a layered fashion [14]. Wang et al. share the weights of the network according to the degrees of nodes [15]. Tang et al. use different parameter sets across iterations but share them among messages in a single iteration [16]. The same approach is also applied to NNMS [17]. These NBP-like decoders[1] mainly reduce the number of parameters to train by weight sharing in many ways. Some simply share weights on the same edge across different layers or share weights on different edges within the same layer. Additionally, utilizing implicit structural properties within codes, such as cyclic permutations and edge expansion, not only can ease the training difficulty, but also may enhance the decoding performance of codes with corresponding structures. Other approaches mentioned above have also made much effort to simplify NBP-like models.

Despite the performance gains produced by plain and modified NBP decoders compared to traditional BP decoders, they are currently usually applied to HDPC and short LDPC codes. There is limited application in cases with LDPC codes of longer block lengths (about 1000 or greater). The main reasons for this are as follows. Firstly, the construction of NBP networks associated with parity-check matrices leads to increased complexity with longer block lengths. As the parity-check matrix size grows, more trainable parameters are introduced, escalating the training difficulty. Secondly, in previous work, despite weight-sharing strategies reducing parameters, weights in every hidden layer need to be expanded into a large and sparse matrix for GPU convenience (details will be introduced later), regardless of the number of parameters that actually need to be trained.[2] Thirdly, iterative decoding on longer LDPC codes shows slower convergence [12], which requires deeper NBP networks that are more difficult to train. These three challenges, from the perspectives of training and practical deployment, greatly constrain the applicability of NBP decoders to longer LDPC codes. Finally, to improve the error-floor performance, NBP networks often need to be further deepened, which increases training difficulty and hardware costs [21].

This paper presents a more comprehensive research upon the earlier work [22]. Compared to our previous work that focused only on the design of NBP-like decoders for QC-LDPC codes, this paper extends the class of codes supported under a unified design paradigm to include cyclic LDPC codes. These codes exhibit low error floors [23] and offer advantages in hardware implementation for encoding and decoding [24]. Furthermore, a type of primitive rate-compatible LDPC (PRC-LDPC) code shows rate compatibility with performance comparable to 5G LDPC codes, while maintaining low encoding complexity [25]. Additionally, we conducted a more detailed complexity analysis of the proposed decoder. Lastly, we present extended experimental results. Specifically, compared to our previous study, the main distinctions of this work are as follows:

- Decoder Design: We introduced a revolving NBP-like decoder tailored for cyclic LDPC codes. Notably, we unified the RID-based NBP-like decoder design for both cyclic and QC-LDPC codes into a consistent design framework. The simplified NBP network can be constructed based on cyclic or QC structures, utilizing either the revolving or section-wise revolving mechanism.
- Complexity Analysis: In this paper, complexity specifically refers to two aspects: (1) computational complexity and (2) the memory overhead required to store trainable parameters in matrix form on the GPU during deployment. We further analyzed the computational complexity of the proposed model in decoding cyclic and QC-LDPC codes, comparing it to the traditional BP algorithm and existing NBP-like models. We also examined the GPU memory overhead for storing weight matrices in the NBP-like decoder designed for cyclic LDPC codes.
- Experimental Results: This paper includes decoding results for cyclic LDPC codes and conducts additional experiments and discussions on decoders for QC-LDPC codes. Through more comprehensive simulations, the performance of the proposed decoder is demonstrated in greater depth. We believe these results can provide valuable insights for the future design and optimization of neural network-based decoders.

[1]Some work do not use NBP-like networks for decoding. Instead, they employ general deep learning models, such as transformers, to decode linear codes, and have got some attractive results [18], [19], [20]. However, these models are beyond the scope of this paper, as they adopt entirely different approaches to designing neural network-based decoders.

[2]Though multi-GPU parallelization offers a workaround, our primary goal is to modify the NBP network itself, reducing model complexity for enhanced practicality.

The remainder of the paper is organized as follows. Section II introduces the cyclic structure of cyclic LDPC codes and the section-wise cyclic structure of QC-LDPC codes. It also briefly reviews mainstream NBP-like decoders. Section III introduces the revolving NBP-like decoder for cyclic LDPC codes and analyzes its complexity and memory usage by detailing the matrix-based data processing of NBP networks. Section IV presents the section-wise revolving NBP-like decoder for QC-LDPC codes, analyzing the complexity and memory usage of NNMS through its matrix-based data processing. It also compares the differences in designing decoders for cyclic and QC-LDPC codes using the RID mechanism. Section V shows the simulation results of the proposed decoders. Finally, Section VI concludes the paper.

II. Preliminaries

A. Structures of Cyclic and QC-LDPC Codes

1) Cyclic Structure of a Cyclic LDPC Code: In this paper, we only consider cyclic LDPC codes whose parity-check matrices are each a single circulant. Consider a cyclic LDPC code $\mathcal{C}_{cyc}$ characterized by an $n \times n$ circulant parity-check matrix:

$$\mathbf{H}_{cyc} = \begin{bmatrix} \mathbf{h}_0^T & \mathbf{h}_1^T & \cdots & \mathbf{h}_{n-1}^T \end{bmatrix}^T, \tag{1}$$

where $\mathbf{h}_i$ $(i = 0, 1, \cdots, n-1)$ denotes each row of $\mathbf{H}_{cyc}$. The i-th $(i > 0)$ row can be obtained by cyclically shifting the $(i-1)$-th row by 1 position to the right, and the 0-th row also can be obtained by shifting the $(n-1)$-th row in the same way. If we start from the 0-th row and group every l rows, we can get a sequence of submatrices of $\mathbf{H}_{cyc}$, denoted by $\mathbf{H}_j$ $(j = 0, 1, \cdots)$, each consisting l *consecutive* rows of $\mathbf{H}_{cyc}$:

$$\mathbf{H}_j = \begin{bmatrix} \mathbf{h}_{(jl) \bmod n}^T & \mathbf{h}_{(jl+1) \bmod n}^T & \cdots & \mathbf{h}_{(jl+l-1) \bmod n}^T \end{bmatrix}^T, \tag{2}$$

where l is called the grouping size. Particularly, when l is a factor of n, the submatrices $\mathbf{H}_0, \mathbf{H}_1, \cdots, \mathbf{H}_{n/l-1}$ form a parition of $\mathbf{H}_{cyc}$. We can see that, for $j > 0$, the submatrix $\mathbf{H}_j$ can be obtained by cyclically shifting all the rows of $\mathbf{H}_{j-1}$ to the right by l position, or by cyclically shifting all the rows of $\mathbf{H}_0$ to the right by *jl* positions simultaneously. That is to say, we can generate the succeeding submatrices and the complete $\mathbf{H}_{cyc}$ from only $\mathbf{H}_0$.

2) Section-Wise Cyclic Structure of a QC-LDPC Code: In most of the proposed QC-LDPC codes, the circulant matrices in the parity-check matrix $\mathbf{H}_{qc}$ of a QC-LDPC code are circulant permutation matrices (CPMs) or zero matrices (ZMs). Each row of a CPM or ZM can be obtained by cyclically shifting the preceding row to the right by 1 position. Let $\mathbf{B}$ be an $c \times r$ matrix called the base matrix of a QC-LDPC code. $\mathbf{H}_{qc}$ is an array of CPMs and/or ZMs derived by replacing each non-zero element of $\mathbf{B}$ with a $Z \times Z$ CPM, and by replacing each zero element of $\mathbf{B}$ with a $Z \times Z$ ZM. As a result, $\mathbf{H}_{qc}$ is of size $cZ \times rZ$, and it can be further partitioned as:

$$\mathbf{H}_{qc} = \begin{bmatrix} \mathbf{H}_{qc,0}^T & \mathbf{H}_{qc,1}^T & \cdots & \mathbf{H}_{qc,c-1}^T \end{bmatrix}^T, \tag{3}$$

where $\mathbf{H}_{qc,j}$ $(j = 0, 1, \cdots, c-1)$ is called a row-block of $\mathbf{H}_{qc}$. The row-block $\mathbf{H}_{qc,j}$ consists of Z consecutive rows of $\mathbf{H}_{qc}$, and can also be seen as a row of CPMs and ZMs derived from the j-th row of $\mathbf{B}$.

Definition 1: Let $\mathbf{H}_{qc,j,k}$ be the k-th circulant matrix in $\mathbf{H}_{qc,j}$, beginning with the row $\mathbf{h}_{j,k}$. So the first whole row of row-block $\mathbf{H}_{qc,j}$ is $(\mathbf{h}_{j,0}, \mathbf{h}_{j,1}, \cdots, \mathbf{h}_{j,r-1})$, containing r *sections*. We refer to the operation of cyclically shifting the row *within the sections* simultaneously as *section-wise* cyclically shifting.

By applying section-wise right cyclically shifting to the current row to generate the next row $Z-1$ times, each by 1 position, the complete j-th row-block is assembled. Therefore, if we construct a simpler $c \times rZ$ matrix $\mathbf{H}^*$ by just extracting the first row of each row-block, all the rows of the original complete $\mathbf{H}_{qc}$ will be generated by simply performing section-wise cyclically shifting on each row of $\mathbf{H}^*$.

B. NBP-Like Decoders

As described earlier, the skeleton of NBP network is derived from the Tanner graph of a linear code. Consider a linear code of block length n, represented by its parity-check matrix $\mathbf{H}$. For the v-th VN and the c-th CN in the Tanner graph, there is an edge (v, c) connected between them if and only if the corresponding element in $\mathbf{H}$ equals 1. In the NBP network, a hidden layer contains $|E|$ nodes, representing each distinct edge, where $|E|$ is the number of edges in the Tanner graph. The input layer and the output layer both consist of n nodes. Consider an NBP network containing $2T$ hidden layers, an input layer, and an output layer, which is equivalent to carrying out T iterations of the traditional decoding algorithm.

The NBP network is usually fed a log-likelihood ratio (LLR) vector as input. Given an original codeword $\mathbf{x}$ and the noisy received signal $\mathbf{y}$ after transmitting $\mathbf{x}$ through a channel, the LLR of the v-th bit in $\mathbf{x}$ is defined as $L_v = \ln \frac{\Pr(x_v=0|y_v)}{\Pr(x_v=1|y_v)}$. Two hidden layers are involved in each iteration. During the iteration t, the first hidden layer updates messages outgoing from VNs after the iteration $t-1$:

$$L_{v\to c}^{(t)} = w_v^{(t)} L_v + \sum_{c' \in \mathcal{N}(v)\backslash c} w_{c'\to v, v\to c}^{(t)} L_{c'\to v}^{(t-1)}, \tag{4}$$

where $\mathcal{N}(v)$ denotes the neighboring set of v and $\mathcal{N}(v)\backslash c$ denotes the neighboring set of v except c. Note the neighboring set of a VN only consists of CNs. The parameters, $w_v^{(t)}$ and $w_{c'\to v, v\to c}^{(t)}$, are both trainable weights. Then the second hidden layer in the iteration t updates messages outgoing from CNs:

$$L_{c\to v}^{(t)} = 2 \tanh^{-1} \left(\prod_{v' \in \mathcal{N}(c)\backslash v} \tanh \left(\frac{L_{v'\to c}^{(t)}}{2} \right) \right), \tag{5}$$

where $\mathcal{N}(c)\backslash v$ denotes the neighboring set of c except v. After going through the $2T$-th iteration, the updated messages will be aggregated for each VN as:

$$o_v = L_v + \sum_{c' \in \mathcal{N}(v)} w_{c'\to v, v}^{out} L_{c'\to v}^{(T)}, \tag{6}$$

where $w_{c'\to v, v}^{out}$ is a trainable weight.

The variants of the original NBP, commonly referred to as NNMS and NOMS, modify the calculation of CN messages

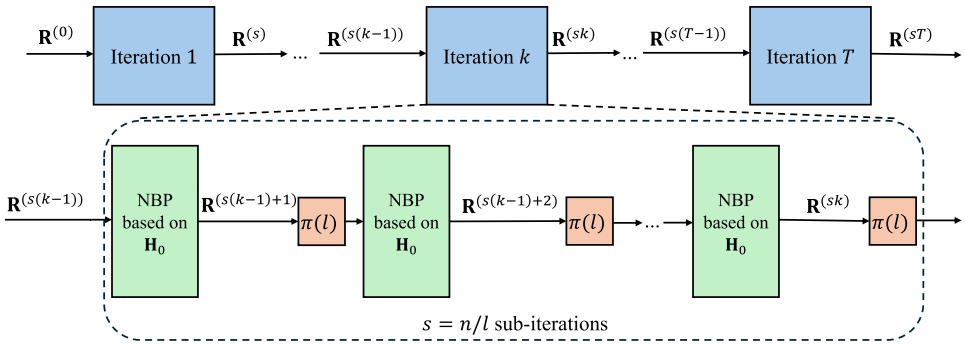


Fig. 1. The overall framework of revolving NBP-like decoders for cyclic LDPC codes, where $\pi(l)$ denotes cyclically shifting the vector to the left by l positions. In this figure, we adopt an NBP network for decoding in each sub-iteration. For clarity, we present the entire framework at two levels: "iteration" and "sub-iteration". In practice, there is no need to partition the sequence of decoders strictly according to the level "iteration".

in (5). Specifically, the NNMS network updates CN messages as

$$L_{c\to v}^{(t)} = w_{c\to v}^{(t)} \left[\prod_{v'\in\mathcal{N}(c)\backslash v} \operatorname{sign}\left(L_{v'\to c}^{(t)}\right)\right] \times \min_{v'\in\mathcal{N}(c)\backslash v} \left|L_{v'\to c}^{(t)}\right|, \tag{7}$$

where $w_{c\to v}^{(t)}$ is the trainable weight in place of the fixed scaling factor used in the NMSA. The NOMS network updates CN messages as

$$L_{c\to v}^{(t)} = \left[\prod_{v'\in\mathcal{N}(c)\backslash v} \operatorname{sign}\left(L_{v'\to c}^{(t)}\right)\right] \times \max\left\{\min_{v'\in\mathcal{N}(c)\backslash v} \left|L_{v'\to c}^{(t)}\right| - \beta_{c\to v}^{(t)}, 0\right\}, \tag{8}$$

where $\beta_{c\to v}^{(t)}$ is the trainable parameter in place of the fixed offset used in the OMSA. Please note that the term "NBP-like networks" is employed to collectively refer to NBP, NNMS, and NOMS networks in this paper. In addition, one important advantage of NMSA and OMSA is that they can directly take the received signal as input without the need for noise estimation to obtain its LLR [1], which makes them more practical for applications. Consequently, NNMS networks and NOMS networks can also leverage this feature.

III. Revolving NBP-Like Decoders for Cyclic LDPC Codes

A. The Framework of Revolving NBP-Like Decoders

As introduced in Section II-A.1, when the grouping size l is a factor of n, the sequence of $s = n/l$ submatrices, $\mathbf{H}_0, \mathbf{H}_1, \cdots, \mathbf{H}_{s-1}$, can be derived from solely $\mathbf{H}_0$ by cyclic shifts, and form the complete cyclic parity-check matrix $\mathbf{H}_{cyc}$. This implies that if we perform a message updating process based on $\mathbf{H}_0, \mathbf{H}_1, \cdots, \mathbf{H}_{s-1}$ successively, then a complete iteration based on $\mathbf{H}_{cyc}$ is accomplished equivalently. We refer to updating messages based on a submatrix as a sub-iteration, so an iteration covering $\mathbf{H}_{cyc}$ consists of s sub-iterations based on s submatrices.

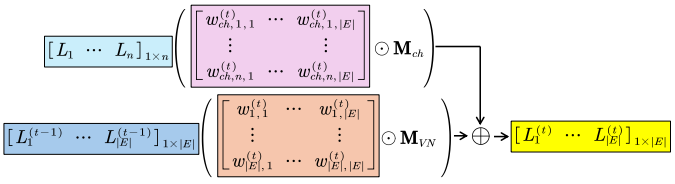
The proposed framework of revolving NBP-like decoders for cyclic LDPC codes is presented in Fig. 1. Instead of building a neural network based on the original $\mathbf{H}_{cyc}$, we choose to build its simplified version using $\mathbf{H}_0$. Then, using the simplified NBP network, we can execute message updating operations, named a *sub-iteration* of the decoding process, which not only reduces the computational load when updating messages, but also alleviates the burden of the GPU memory for easier deployment. Besides, considering the cyclic shift relationship between $\mathbf{H}_0$ and the subsequent submatrices, there is no need to construct a new NBP network for each individual submatrix: after cyclically shifting the output of the current NBP network to the left by l positions, we feed the shifted vector to the next network as input. This process is equivalent to decoding using the next submatrix in the sequence, but reusing the same model contributes to further saving the GPU memory consumption.

Algorithm 1 Revolving NBP Decoders for Cyclic LDPC Codes

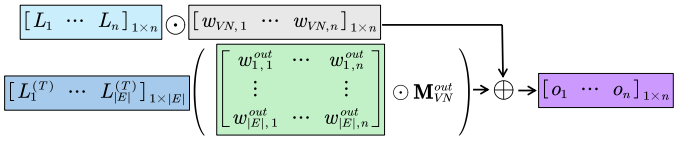
Input: LLR vectors $\mathbf{L}$
Output: Updated reliablility vector $\mathbf{R}^{(sT)}$
1: $\mathbf{R}^{(0)} \leftarrow \mathbf{L}^{(0)}$ // Initialization
2: **for** $t \leftarrow 1$ **to** sT **do**
3: $\quad \mathbf{R}^{(t)} \leftarrow \text{NBP\_Decoder}(\mathbf{R}^{(t-1)})$
4: $\quad \mathbf{R}^{(t)} \leftarrow \pi(\mathbf{R}^{(t)}, l)$ // cyclic left shift by l positions
5: **end for**

The general procedure of the NBP decoders for cyclic LPDC codes is outlined in Algorithm 1. For sub-iterations that involve an NBP network, we initialize the reliability vector $\mathbf{R}^{(0)}$ as the LLR vector of the received signal $\mathbf{y}$. While in the case that we use an NNMS or NOMS network, initializing $\mathbf{R}^{(0)}$ as $\mathbf{y}$ is recommended, following the way in the NMSA and OMSA. During the sub-iteration t, the reliability vector from the last loop $\mathbf{R}^{(t-1)}$ is sent into the NBP decoder and a new vector $\mathbf{R}^{(t)}$ is produced. The NBP decoders across sub-iterations share the same set of weights. In other words, we only need to train a single network consisting of two hidden layers and an output layer rather than sT different networks, which is like the NBP of RNN architecture in [8]. Additionally, updating messages with this single network each time based on $\mathbf{H}_0$ is equivalent to one sub-iteration of the RID scheme in [5].

Note that the backpropagation of gradients during training is supported automatically thanks to modern deep learning



(a) The process of updating a batch of VN message.



(b) The process of calculating a new reliability vector (or output vector) in the output layer of the NBP network.

Fig. 2. The diagrams of data processing of a vanilla NBP network in a matrix-based manner on GPU, where the operators $\odot$ and $\oplus$ denote element-wise matrix multiplication and addition respectively. The batch size is set to 1 for clarity.

TABLE I
COMPARISON ON COMPUTATIONAL COMPLEXITY OF DIFFERENT (N)BP DECODERS

	Decoder	SPA	SPA (Mat. form)	NBP	NBP-RID (ours)
CN update	MUL	$2\|E\|d_c$	$4\|E\|^2-\|E\|$	$4\|E\|^2-\|E\|$	$4(\|E\|/s)^2-\|E\|/s$
	$\tanh(\cdot)$	$\|E\|(d_c-1)$	$\|E\|^2$	$\|E\|^2$	$\|E\|^2/s^2$
	$\tanh^{-1}(\cdot)$	$\|E\|$	$\|E\|$	$\|E\|$	$\|E\|/s$
VN update	MUL	$\|E\|d_v$	$\|E\|(n+\|E\|)$	$2\|E\|(n+\|E\|)$	$2(\|E\|/s)(n+\|E\|/s)$
	ADD	$\|E\|(d_v-1)$	$\|E\|(n+\|E\|-1)$	$\|E\|(n+\|E\|-1)$	$(\|E\|/s)(n+\|E\|/s-1)$

tools such as PyTorch [26], regardless of how many times we cyclically shift the reliability vector.

B. Message Updating and Complexity Analysis

Here we analyze the complexity of the original NBP and the revolving NBP decoder, as illustrated in Fig. 2, starting with how a hidden layer in the NBP networks updates messages, ending with an example for a more detailed explanation.

1) Message Updating Process of NBP: Consider a cyclic LDPC code $\mathcal{C}_{cyc}$ of block length n characterized by the parity-check matrix $\mathbf{H}_{cyc}$ whose row weight is γ. We denote the NBP decoder based on the complete $\mathbf{H}_{cyc}$ as $\mathcal{D}$, and the NBP decoder based on the first l rows of $\mathbf{H}_{cyc}$, namely $\mathbf{H}_0$, as $\mathcal{D}_0$. For $\mathcal{D}$, the dimension of a hidden layer is $|E|=n\gamma$, where $|E|$ is the number of 1-entries in $\mathbf{H}_{cyc}$. From the point view of the Tanner graph, the nodes in a hidden represent $|E|$ different edges. Therefore, when we refer to a node in the hidden layer, we are also referring to the unique corresponding edge in the Tanner graph. Moreover, connections between nodes in the h-th hidden layer and the $(h+1)$-th hidden layer depend on whether they are connected to the same VN or CN in the Tanner graph, which adhere to the following rules:

1) If the h-th hidden layer updates CN messages, a node in it will be connected to the nodes in the $(h+1)$-th hidden layer sharing the same CN except itself.
2) If the h-th hidden layer updates VN messages, a node in it will be connected to the nodes in the $(h+1)$-th hidden layer sharing the same VN except itself.

Such a relationship can be represented by an $|E|\times|E|$ mask matrix $\mathbf{M}_{VN}$ (or $\mathbf{M}_{CN}$) for a hidden layer. If the a-th node of a hidden layer is connected to the b-th node of the next layer, the element in the b-th row and the a-th column $M_{VN,b,a}$ (or $M_{CN,b,a}$) is set to 1, otherwise $M_{VN,b,a}$ (or $M_{CN,b,a}$) is set to 0.

Next, we will elaborate on the operation of the hidden layers responsible for updating VN messages, excluding the hidden layers for CN message updating because there are no trainable weights introduced in these layers. When a vector $\mathbf{L}^{(t-1)}$ of length $|E|$, denoting $|E|$ different messages as in (4), goes through the hidden layer in the iteration t, the calculation of (4) can be formulated as

$$\mathbf{L}^{(t)}=\mathbf{L}\left(\mathbf{W}_{ch}^{(t)}\odot\mathbf{M}_{ch}\right)+\mathbf{L}^{(t-1)}\left(\mathbf{W}^{(t)}\odot\mathbf{M}_{VN}\right), \quad (9)$$

where $\odot$ denotes the element-wise multiplication for two matrices or vectors. $\mathbf{W}_{ch}^{(t)}$ is an $n\times|E|$ matrix including the set of trainable weights $\{w_v^{(t)}\}$, and $\mathbf{W}^{(t)}$ is the $|E|\times|E|$ matrix including the set of trainable weights $\{w_{c'\to v,v\to c}^{(t)}\}$, both of which are in a very sparse form. $\mathbf{L}$ is the LLR vector of length n obtained from the channel output. $\mathbf{M}_{ch}$ is the $n\times|E|$ mask matrix, where the b-th row and the a column element $M_{ch,b,a}$ denotes if the a-the edge of the Tanner graph is connected to the b-th VN. In the end, the output of the hidden layer, $\mathbf{L}^{(t)}$, is obtained. Finally, the output layer executes like the hidden layer described above with a little difference:

$$\mathbf{o}=(\mathbf{w}_{VN}\odot\mathbf{L})+\mathbf{L}^{(T)}\left(\mathbf{W}^{out}\odot\mathbf{M}_{VN}^{out}\right), \quad (10)$$

where the output vector $\mathbf{o}$ contains all the elements obtained as (6), which is also referred to the reliability vector $\mathbf{R}$ produced by an NBP decoder in a sub-iteration. $\mathbf{w}_{VN}$ is the weight vector of n VNs (usually fixed to the all-ones vector), and $\mathbf{W}^{out}$ is the $|E|\times n$ matrix including the set $\{w_{c'\to v,v}^{out}\}$. $\mathbf{M}_{VN}^{out}$ is the $|E|\times n$ mask matrix, where $M_{VN,b,a}^{out}$ denotes if the b-th edge is connected to the a-th VN. There are two primary distinctions between $\mathbf{M}_{VN}$ and $\mathbf{M}_{VN}^{out}$ that warrant attention: first, their shapes are different; second, for a given VN, $\mathbf{M}_{VN}$ requires the exclusion of a message from one edge, whereas $\mathbf{M}_{VN}^{out}$ does not exclude messages from any edges.

2) Complexity Analysis and Memory Consumption: The computational complexities of different decoders are listed in Table I, where d_c $(=\gamma)$ and d_v denote the degree of a CN and a VN, respectively. For our proposed decoder (denoted as NBP-RID), the table presents the number of operations required for a single sub-iteration, whereas for other decoders, it shows the number of operations required for an iteration. Since NBP-RID only uses l rows of the complete parity-check matrix, the number of edges is reduced to $1/s=l/n$ of that in the original

TABLE II
THE GPU MEMORY OCCUPANCY OF THE SPARSE WEIGHT MATRICES OF DIFFERENT NBP DECODERS IN EXAMPLE 1

Decoder	Original	RNN-like	NBP-RID (ours)
GPU memory occupancy estimate	23.5 GB	4.8 GB	0.27 MB

NBP, namely $|E|$. In the NBP decoder, when all trainable parameters are fixed to 1, it corresponds to the traditional SPA. Therefore, we provide the computational complexities for both the traditional version implemented directly and the matrix-based version similar to NBP (denoted as Mat. form). Compared to NBP, the matrix-based SPA omits the element-wise multiplication of the mask matrices $\mathbf{M}_{VN}$ and $\mathbf{M}_{ch}$ with their respective weight matrices when updating VN messages.

It should be noted that we provide the matrix-based version of the traditional algorithm not to encourage the straightforward implementation of it, but to establish a comparison bridge between neural network-based decoders (mainly using GPUs) and traditional decoders (mainly using CPUs and RAM, or other hardware platforms). As shown in the Table I, traditional algorithms still have an advantage in computational complexity, as they perform fewer operations. However, by introducing additional learnable parameters, NBP-like decoders can theoretically mitigate the negative effects of short cycles in the Tanner graph on message passing, using the extra computational cost to improve decoding performance [8], [12].

As we emphasized before, regardless of how the sizes of set $\{w_{c'\to v,v\to c}^{(t)}\}$ and $\{w_v^{(t)}\}$ are reduced by weight sharing, the sparse matrix $\mathbf{W}^{(t)}$ and $\mathbf{W}_{ch}^{(t)}$ are necessary when we deploy the NBP network on a GPU to speed up the data processing in the neural network by efficient matrix operations, which is always stored in the GPU memory. Even when we assign no weights to messages to carry out the traditional BP algorithm, two special matrices that only contain 1-entries and 0-entries are still required when using the GPU.

For $\mathcal{D}_0$, the dimension of a hidden layer is $|E_0| = l\gamma$, where $|E_0|$ is the number of 1-entries in $\mathbf{H}_0$. In the hidden layer that updates VN messages in the iteration t, the sparse weight matrices, $\mathbf{W}_0^{(t)}$ and $\mathbf{W}_{ch,0}^{(t)}$, are of size $|E_0|\times|E_0|$ and $n\times|E_0|$ respectively. It is clear that the number of elements stored in the GPU memory now is $1/s^2 = (l/n)^2$ and $1/s$ of those of the original NBP network, respectively. Additionally, the reduced sizes of learnable weights $\{w_{c'\to v,v\to c}^{(t)}\}$ and $\{w_v^{(t)}\}$ are both $1/s$ of the original sizes. The following example provides an intuitive comparison of the GPU memory consumption of trainable parameters in *neural network-based* decoders.

Example 1: Consider the (1057,813) cyclic PG-LDPC given in Example 10.11 of [1, Chapter 10, p. 456]. The block length is $n = 1057$ and the parity-check matrix $\mathbf{H}_{PG}$ is a single 1057×1057 circulant with both row weight and column weight $\gamma = 33$. The total number of edges in the Tanner graph based on $\mathbf{H}_{PG}$ is $|E| = 1057\times33$.

If we build an NBP decoder $\mathcal{D}_{PG}$ based on $\mathbf{H}_{PG}$, including $2T = 10$ hidden layers, which is a common choice as in [8]. The trainable parameters of a typical NBP model are all stored as default 32-bit (4-byte) floating-point numbers in the GPU memory. When the model $\mathcal{D}_{PG}$ is decoding a codeword, the space occupied by sparse weight matrices is

$$\left(\frac{4\text{ bytes}}{1024^3}\right)\Big(\underbrace{T\times\left(|E|^2+n|E|\right)}_{\text{hidden layers}}+\underbrace{n+n|E|}_{\text{output layer}}\Big)\approx 23.5\text{ GB}. \tag{11}$$

Please note that the calculation in (11) is a very rough estimate excluding other data stored in the GPU memory, such as gradients, inputs, and various intermediate data. Nevertheless, it is sufficient to illustrate our points. In comparison, the memory of an NVIDIA GeForce RTX 3090 is 24 GB. Hence it is difficult to deploy the NBP decoder $\mathcal{D}_{PG}$ directly on a single GPU without any modification.

One can choose to adopt the weight sharing method to reuse the trainable weights across decoding iterations, also known as the RNN-like architecture mentioned before. In this case, T is reduced to 1, and the result of (11) becomes about 4.8 GB, which seems to be acceptable for a single GPU.

However, it is feasible to further alleviate the occupancy of the GPU memory using the revolving NBP decoder. We can derive a submatrix $\mathbf{H}_{PG,0}$ by extracting the first l rows of $\mathbf{H}_{PG}$. If $l = 1$, another NBP $\mathcal{D}_{PG,0}$ based on $\mathbf{H}_{PG,0}$ is much simpler than $\mathcal{D}_{PG}$, consisting of two hidden layers and an output layer. The number of edges in the Tanner graph derived from $\mathbf{H}_{PG,0}$ is $|E_0| = 1\times33 = |E|/1057$.As is shown in Table II, the revolving decoder only occupies about 0.27 MB space by calculating as (11). This contributes to substantial decreases in both GPU memory consumption and computational complexity, enabling the deployment of an NBP decoder capable of decoding longer LDPC codes on a single GPU.

3) The Selection of Grouping Size: A natural question arises regarding the selection strategy for the hyper-parameter l. Through experimental analysis, we empirically recommend limiting the selection of l to a relatively small range. To supplement our empirical results, we provide a formal analysis of l selection based on the generalization bound. This theoretical framework is used to analyze the parameter's effect on the decoder's generalization capability—specifically, the gap between its training and true performance [27]. Therefore, this analysis supports and validates our experimental findings from a theoretical standpoint.

Problem Statement: The main goal of this part is to analyze the generalization gap $\mathcal{R}_{\text{BER}}(f)-\hat{\mathcal{R}}_{\text{BER}}(f)$, where $\hat{\mathcal{R}}_{\text{BER}}(f)\triangleq\frac{1}{m}\sum_{s=1}^{m}l_{\text{BER}}\left(f\left(\mathbf{L}_s\right),\mathbf{x}_s\right)$ is the the empirical risk of f, and $\mathcal{R}_{\text{BER}}(f)\triangleq\mathbb{E}_{\mathbf{L},\mathbf{x}}\left[l_{\text{BER}}(f(\mathbf{L}),\mathbf{x})\right]$, l_{BER} is the bit error rate (BER) loss [28].

Notations: The proposed decoder, denoted as $f(\cdot)$, is characterized by four classes of sparse weight matrices: (a) $\mathbf{W}_{ch}^{(t)}\in\mathbb{R}^{n\times|E|}, t=1\ldots,T$, (b) $\mathbf{W}^{(t)}\in\mathbb{R}^{|E|\times|E|}, t=1\ldots,T$, (c) $\mathbf{W}^{out}\in\mathbb{R}^{|E|\times n}$, and (d) $\mathbf{W}_{VN}\in\mathbb{R}^{n\times n}$. Note that $\mathbf{W}_{VN}=\text{diag}(w_1,\ldots,w_n)$ represents the diagonal matrix form of the vector $\mathbf{w}_{VN}=(w_1,\ldots,w_n)$ as defined in (10). Additionally, for simplicity, the weight matrices here have implicitly applied the mask operation described in (9) and (10), such that the matrix entries are zero at positions where weights are not required. In practice, we train a decoder on the dataset $\{(\mathbf{L}_s,\mathbf{x}_s)\}_{s=1}^m$ consisting of m pairs of LLR vectors and corresponding codewords.

Theorem 1: For any $\delta \in (0,1)$, with probability at least $1-\delta$, the generalization gap for any NBP decoder $f \in \mathcal{F}_T$ can be upper bounded as follows:

$$\mathcal{R}_{\mathrm{BER}}(f) - \hat{\mathcal{R}}_{\mathrm{BER}}(f) \leq \frac{4}{m} + \sqrt{\frac{\log(1/\delta)}{2m}} + 12\sqrt{\frac{(ld_c d_v T + d_v + 1)}{m}\,\mathrm{term}}, \tag{12}$$

where $\mathrm{term} = \log\left(8\sqrt{m\min(ld_c, n)}b_L(T+1)^2\right) + (T+1)\log\left(\sqrt{n}wd_v\right)$, $\mathcal{F}_T$ is the function class of NBP decoders with T iterations, d_c is the CN degree of a cyclic LDPC code, d_v is the maximum VN degree of $\mathbf{H}_0$ by using l rows of the complete matrix for decoding, T is the number of iterations, and n is the block length, b_L and w are the upper bounds on the value of each input LLR and the weights in the decoder, respectively.

The detailed proof of Theorem 1 is presented in the Supplementary Material and here we present the key insight from the theorem above. The generalization gap scales with the grouping size as:

$$\begin{cases} \mathcal{O}(\sqrt{ld_v\left(\log(\sqrt{n}) + \log(\sqrt{n}d_v)\right)}) & l \leq n/d_c \\ \mathcal{O}(\sqrt{ld_v\left(\log(\sqrt{ld_c}) + \log(\sqrt{n}d_v)\right)}) & l > n/d_c \end{cases} \tag{13}$$

Note that d_v is also controlled by l. Since when $l > n/d_c$, it follows that $ld_c > n$, the generalization gap bound grows faster in the larger l regime. Therefore, it is easier to train an NBP decoder with stronger generalization capability when $l \leq n/d_c$, resulting in a smaller gap between empirical risk and true risk.

IV. Section-Wise Revolving NBP-Like Decoders for QC-LDPC Codes

A. The Design of Section-Wise Revolving NBP-Like Decoders

The parity-check matrix $\mathbf{H}_{qc}$ of a QC-LDPC code expanded from the $c \times r$ base matrix $\mathbf{B}$ with $Z \times Z$ CPMs and ZMs, as is mentioned in Section II-A.2, features a section-wise cyclic property between any two rows within a row-block.

Definition 2: Let the $c \times rZ$ matrix $\mathbf{H}^*$ be constructed by just extracting the top row of each row-block of $\mathbf{H}_{qc}$ and stacking them up in order. This submatrix $\mathbf{H}^*$ of $\mathbf{H}_{qc}$ is called the decoding matrix of the proposed decoder.

Given the decoding matrix $\mathbf{H}^*$ consisting of the first rows of c row-blocks, $\mathbf{H}_{qc}$ can be covered completely by section-wise cyclically right shifting c rows of $\mathbf{H}^*$ by 1 position simultaneously, repeated $Z-1$ times. We refer to the decoding based on $\mathbf{H}^*$ as a *sub-iteration*. Before using the reliability vector in the next sub-iteration each time, we cyclically shift it to the left by 1 position in a *section-wise* manner, which differs from the cyclic shift operation when decoding cyclic LDPC codes. Finally, we can readily be aware that Z sub-iterations cover all the message exchanges based on $\mathbf{H}_{qc}$ in one iteration equivalently. Building upon this, performing ZT sub-iterations based on $\mathbf{H}^*$ is an alternative to conducting T iterations based on $\mathbf{H}_{qc}$.

Algorithm 2 Revolving NNMS Decoders for QC-LDPC Codes

Input: Channel output $\mathbf{y}$
Output: Updated reliability vector $\mathbf{R}^{(ZT)}$

1: $\mathbf{R}^{(0)} \leftarrow \mathbf{y}$
2: $\mathbf{L}_{ex}^{(t)} \leftarrow \mathbf{0}, \quad t = 0, -1, \cdots, 1-Z$
3: **for** $t \leftarrow 1$ **to** ZT **do**
4: /* Updating VN messages */
5: $\mathbf{R}^{(t-1)} \leftarrow \mathbf{R}^{(t-1)} - \mathbf{L}_{ex}^{(t-Z)}$
6: /* Updating CN messages*/
7: **for** each CN c **do**
8: update $L_{c\to v}^{(t)}$ as (7)
9: **end for**
10: /* Updating $\mathbf{L}_{ex}^{(t)}$ */
11: **for** each VN v **do**
12: $L_{ex,v}^{(t)} \leftarrow \sum_{c' \in \mathcal{N}(v)} w_{c'\to v,v}^{(t)} L_{c'\to v}^{(t)}$
13: **end for**
14: $\mathbf{R}^{(t)} \leftarrow \mathbf{R}^{(t-1)} + \mathbf{L}_{ex}^{(t)}$
15: $\mathbf{R}^{(t)} \leftarrow \pi_{sec}(\mathbf{R}^{(t)}, 1)$
16: **end for**

We present the specific architecture of the section-wise revolving NNMS decoder in the sub-iteration t in Fig. 3, and further describe the overall procedure in Algorithm 2. In Algorithm 2, $L_{ex,v}^{(t)}$ denotes the weighted sum of messages outgoing from CNs and received at the v-th VN in the sub-iteration t, forming the vector $\mathbf{L}_{ex}^{(t)}$ of length n. Also, the initializations are denoted as $\mathbf{L}_{ex}^{(0)}, \ldots, \mathbf{L}_{ex}^{(1-Z)}$ for convenience. Note that the set of neighboring CNs of the v-th VN, denoted as $\mathcal{N}(v)$, only includes those in the Tanner graph derived from $\mathbf{H}^*$ in Algorithm 2.

Furthermore, in lines 7-15 of Algorithm 2, we adopt an NNMS network, updating CN messages as (7) once, so the NNMS network consists of an input layer for calculating current VN messages transmitted on each edge, a hidden layer for updating CN messages, and an output layer for producing the modified reliability vector. For an enhanced error-correction capability, we can integrate NNMS and NOMS methods as

$$L_{c\to v}^{(t)} = \left[\prod_{v' \in \mathcal{N}(c)\setminus v} \mathrm{sign}\left(L_{v'\to c}^{(t)}\right)\right] \times \mathrm{ReLU}\left(w_{c\to v}^{(t)} \times \min_{v' \in \mathcal{N}(c)\setminus v}\left|L_{v'\to c}^{(t)}\right| - \beta_{c\to v}^{(t)}\right), \tag{14}$$

where the operation $\mathrm{ReLU}(\cdot)$ is defined as $\mathrm{ReLU}(x) = \max(x, 0)$. By introducing more learnable parameters, the degrees of freedom for optimization are expanded [12]. As a result, this design provides a heightened potential to generalize our proposed approach across other classes of QC-LDPC codes, different from the previous approach in [5] which is constrained to the algebraic QC-LDPC codes.

In general, this decoder can be regarded as an enhanced implementation of CPM-RID [5], wherein some tradi-

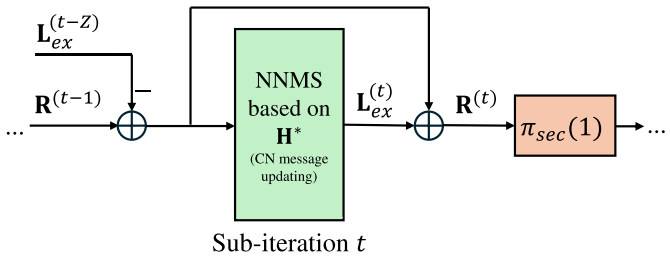


Fig. 3. The architecture of the section-wise revolving NNMS decoder in the sub-iteration t for QC-LDPC codes, where $\pi_{sec}(1)$ denotes section-wise cyclically shifting the vector to the left by 1 position.

tional operations are replaced with an NNMS network. The pseudocode from the 6-th line to the 16-th line in Algorithm 2 describes how the NNMS work updates CN messages in a convolution manner for clarity and simplicity, while in practice the relative operations are executed in matrix form. We will elaborate on this further in Section IV-C.1.

B. Comparison Between Revolving NBP-Like Decoders for Cyclic and QC-LDPC Codes

Although the methodology of designing new decoders for QC-LDPC codes is similar to that in Section III intuitively and generally, we should point out that it is necessary to dive into the way that the network updates messages in each sub-iteration. The reason for this is mainly 2-fold:

1) Fewer Redundant Rows of the Parity-Check Matrix: The parity-check matrix $\mathbf{H}_{cyc}$ of a cyclic LDPC code has large row redundancy, whereas the parity-check matrix $\mathbf{H}_{qc}$ of a QC-LDPC code has small or zero row redundancy in most cases. Consequently, for a cyclic LDPC code, the negative impact of messages can be mitigated easily by sufficient redundant rows, even when using a simplified matrix $\mathbf{H}_0$ for decoding. However, the poor redundancy poses a challenge to decoding the QC-LDPC code to minimize the performance degradation as much as possible based on the simplified matrix $\mathbf{H}^*$.

2) Fewer Edges of the Tanner Graph: The density of the parity-check matrix of a cyclic LDPC code is relatively much higher than a QC-LDPC code, and the same holds true for their respective alternative $\mathbf{H}_0$ and $\mathbf{H}^*$. As a result, there are fewer edges in the Tanner graph based on $\mathbf{H}^*$ than the Tanner graph based on $\mathbf{H}_0$. Compared to the cyclic LDPC code, the reliability of messages of the QC-LDPC code is more fragile and more susceptible to disruption during message passing on an incomplete Tanner graph. Hence, it is essential to design a more refined decoder tailored for QC-LDPC codes.

To address the performance degradation when the reliability vector is directly used for a sub-iteration, we design a new pipeline for the decoders like that in [5]. As is displayed in Fig. 3, in the t-th sub-iteration, before carrying out CN message updating, the messages from CNs $\mathbf{L}_{ex}^{(t-Z)}$ in the $(t-Z)$-th sub-iteration are subtracted from the current reliability vector $\mathbf{R}^{(t-1)}$. As a result, the message sent from a VN to its adjacent CNs is the difference between the updated reliability and messages from all its adjacent CNs in the previous corresponding sub-iteration, leading to a noticeable performance improvement that was also mentioned in [5]. Additionally, the skip connection in each sub-iteration contributes to better performance, which is widely applied in deep learning to obtain a more powerful neural network [29].

C. Message Updating and Complexity Analysis

In this section, we compare the revolving NNMS decoder with the vanilla NNMS decoder defined as (4), (6) and (7). We commence by introducing the process of data handling on GPU, and conclude with a complexity analysis and comparison between these two networks.

1) Matrix-Based Processing for Message Updating: In deep learning, models are deployed on GPUs to leverage their superiority in matrix computations, thus data processing is conducted through matrix-based operations. For a more intuitive illustration, we have depicted the steps in Fig. 4 using the NNMS network.

Firstly, we need to subtract CN messages from the reliability vector as described in Section IV-A, which is performed in the input layer of the NNMS network. This step is naturally conducted in a matrix-based way. Then each element of the vector serves as the message outgoing from each VN. One can find that this step is distinct from that in the conventional BP, because messages outgoing from a VN are the same.

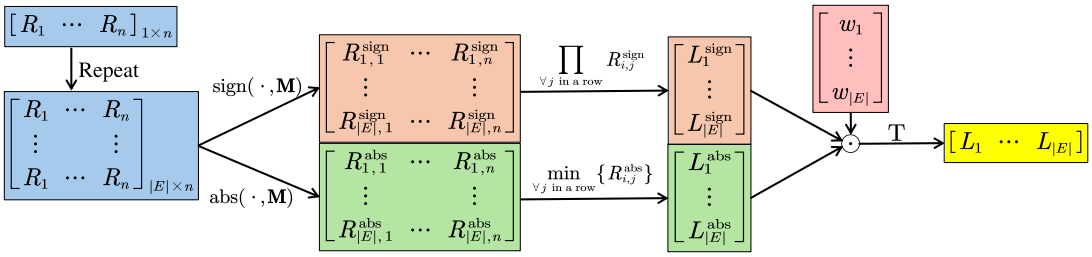
Secondly, to obtain $|E|$ different messages transmitted along edges of the Tanner graph as (7), we need $|E|$ copies of the reliability vector, and then stack them vertically as is displayed in Fig. 4a, forming an $|E| \times n$ matrix $\mathbf{R}$. Specifically, the i-th row of $\mathbf{R}$ is used to calculate the message on the i-th edge (v, c) outgoing from CN c. Since messages from VNs that are not connected to CN c directly should be omitted in the i-th row of $\mathbf{R}$, we use an $|E| \times nmask$ matrix $\mathbf{M}$ to describe relationships between the i-th edge and each VN, based on the decoding matrix. For the i-th edge (v, c), one of n elements in the i-th row of $\mathbf{M}$ shows if a VN is directly connected to CN c. Specifically, the value of the element $M_{i,j}$ in the i-th row and j-th column of $\mathbf{M}$ is defined as

$$M_{i,j} = \begin{cases} 1, & H_{c,j} = 1 \text{ and } v \neq j \\ 0, & \text{otherwise .} \end{cases} \tag{15}$$

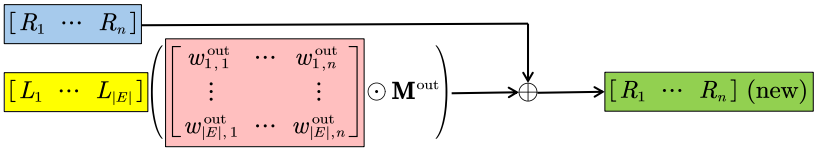
Then, for each CN message (or each row of $\mathbf{R}$), we need to compute the product of the sign and the minimum absolute value of the *unmasked* elements based on $\mathbf{M}$. These two operations are denoted as $\text{sign}(\cdot, \mathbf{M})$ and $\text{abs}(\cdot, \mathbf{M})$ respectively. Specifically, they are defined as

$$\text{sign}(R_{i,j}, \mathbf{M}) = \begin{cases} \text{sign}(R_{i,j}), & M_{i,j} = 1 \\ +1, & M_{i,j} = 0, \end{cases} \tag{16}$$

$$\text{abs}(R_{i,j}, \mathbf{M}) = \begin{cases} \text{abs}(R_{i,j}), & M_{i,j} = 1 \\ +\infty, & M_{i,j} = 0, \end{cases} \tag{17}$$



(a) The process of updating a batch of CN message, where $\mathrm{sign}(\cdot, \mathbf{M})$ calculates the sign of each element based on the mask $\mathbf{M}$, $\mathrm{abs}(\cdot, \mathbf{M})$ calculates the magnitude of each element based on the mask $\mathbf{M}$, and "T" denotes matrix transposition.



(b) The process of calculating a new reliability vector in the output layer of the NNMS network.

Fig. 4. The diagrams of data processing of an NNMS network in a matrix-based manner on GPU, where the operators $\odot$ and $\oplus$ denote element-wise matrix multiplication and addition respectively. The batch size is set to 1 for clarity.

TABLE III
COMPARISONS ON COMPUTATIONAL COMPLEXITY OF NMSA, NNMS AND OUR PROPOSED DECODERS FOR QC-LDPC CODES WHEN UPDATING CN MESSAGES

	NMSA	NMSA (Mat. form)	NNMS	NNMS-CPM-RID	NNOMS-CPM-RID
MUL	$\|E\|d_c$	$2\|E\|^2+\|E\|$	$2\|E\|^2+\|E\|$	$2(\|E\|/Z)^2+\|E\|/Z$	$2(\|E\|/Z)^2+\|E\|/Z$
ADD	\	\	\	\	$\|E\|/Z$

TABLE IV
COMPARISONS ON WEIGHT COMPLEXITY OF NBP-LIKE DECODERS FOR QC-LDPC CODES

	NBP in [8] (original)	NNMS in [8]	NNMS (RNN)	[12]	NNMS-CPM-RID (ours)
Decoding matrix	$\mathbf{H}_{qc}$	$\mathbf{H}_{qc}$	$\mathbf{H}_{qc}$	$\mathbf{H}_{qc}$	$\mathbf{H}^*$
Weight sharing	\	\	by iteration	by edge type	by iteration
# of weights	$T(n+\|E\|(d_c-1))+n+nd_c$	$T\|E\|+n+nd_c$	$\|E\|+n+nd_c$	$T\cdot 2(\|E\|/Z)$	$\|E\|/Z+nd_c$
Weight comsumption (in matrix form)	$T(n\|E\|+\|E\|^2)+n+n\|E\|$	$T\|E\|+n+n\|E\|$	$\|E\|+n+n\|E\|$	$T\cdot 2\|E\|$	$\|E\|/Z+(\|E\|/Z)n$

where $R_{i,j}$ is the element in the i-th row and j-th column of $\mathbf{R}$. Thus we can easily compute the product of the sign and the minimum absolute value of the *entire* row, excluding the influence of irrelevant elements (messages). After the element-wise multiplication of signs, absolute values, and learnable weights, the CN messages transmitted along $|E|$ edges are obtained simultaneously.

Finally, the output layer produces a new reliability vector as presented in Fig. 4b. The mask matrix in the output layer is denoted as $\mathbf{M}^{\mathrm{out}}$. For the VN v, if there is an edge (v, c) indexed i, the element in the i-th row and v-th column of $\mathbf{M}^{\mathrm{out}}$ is set to 1, otherwise is set to 0. As is vividly depicted in the diagram, the weight matrix is of size $|E| \times n$, which is a very sparse matrix after element-wise multiplied with $\mathbf{M}^{\mathrm{out}}$.

2) Complexity Analysis: The Tanner graph derived from $\mathbf{H}_{qc}$ of size $cZ \times rZ$ consists of $|E|$ edges, while the simplified Tanner graph derived from $\mathbf{H}^*$ consists of $|E|/Z$ edges. For example, consider a regular QC-LDPC code whose row weight is d_c, which is also the degree of each CN in the Tanner graphs based on $\mathbf{H}_{qc}$ and $\mathbf{H}^*$. As the step with the greatest differences in decoding, we compared the computational complexity of updating CN messages for NMSA, NNMS, and the proposed NNMS-CPM-RID and NNOMS-CPM-RID decoders based on (7) and (14), respectively. The numbers of required multiplication and addition operations are summarized in Table III. For our decoders, this corresponds to a sub-iteration, whereas for other decoders, it corresponds to an iteration. We provide both a direct implementation of NMSA based on the formula description and a version that utilizes matrix-based operations by fixing the parameters of NNMS to a predetermined scaling factor. Our proposed decoder requires less computation per message update compared to NNMS.

In Table IV, the number of *different* learnable weights and the memory consumption of weights in the matrix form of several NBP-like decoders are presented. Please note that in practice, there are other intermediate results and gradients for backpropagation stored in the memory and it is quite cumbersome to calculate them completely and explicitly so we omit them here. In summary, the advantages of our proposed decoder are lower computational complexity, lower deployment difficulty, and lower training complexity by extremely simplifying the neural network itself.

TABLE V
DETAILS OF THE HYPER-PARAMETERS AND RELATED CONFIGURATIONS

Item	Settings
Loss	Binary cross entropy (BCE) loss
Optimizer	Adam [31]
Training Set	2×10^6 noisy all-zero codewords under AWGN channel
Training SNR	2.0∼3.0 dB, uniform sampling
Batch Size	1000 when start, increase till 60000
Learning Rate	From 0.001 to 0.00001
GPU	A single NVIDIA GeForce RTX 3090

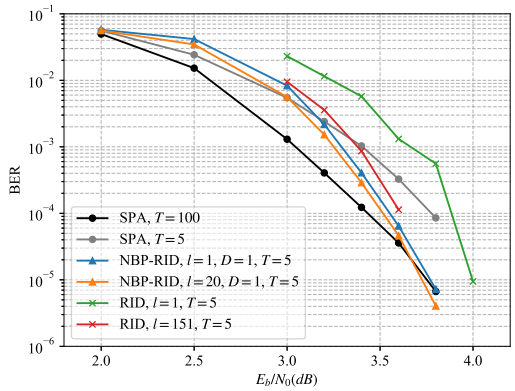


Fig. 5. BER results for (1057, 813) cyclic PG-LDPC code.

V. SIMULATIONS

In this section, we evaluate the performance of the proposed revolving NBP-like decoders for cyclic and QC-LDPC codes. Specifically, we consider traditional decoding methods, including SPA, NMSA, RID [4] for cyclic LDPC Codes, and CPM-RID [5] for QC-LDPC codes, as the primary benchmarks for comparison. To align with the parameter settings in [4] and [5], the scaling factor for RID is set to 0.25, and the scaling factors for NMSA and CPM-RID are both set to 0.75. To train the decoders, we adopted the hyper-parameters and related configurations listed in Table V. It is worth noting that, due to the symmetry of the BP decoding algorithm, the error probability is independent of the transmitted codeword. NBP-like decoders share this property as well. Consequently, the training set can be constructed using only noisy all-zero codewords, an approach widely adopted in previous studies on the training of NBP-like decoders [7], [8], [9], [10], [11], [12], [13], [14], [15], [21], [22], [30].

A. Performance of Revolving NBP Decoders for Cyclic LDPC Codes

We consider the (1057, 813) cyclic PG-LDPC given in Example 1, and present the simulation results in Fig. 5, where the proposed revolving NBP decoder is denoted as NBP-RID. For an NBP-RID decoder, l refers to the grouping size and the number of rows of the submatrix $\mathbf{H}_0$. D represents the number of consecutive message updates performed by the NBP-RID network on the same set of matrix rows. Typically, if $D = 1$, the decoder only performs one two-step message updating in each iteration, so the NBP-RID network only contains two hidden layer for CN message updating and VN message updating based on $\mathbf{H}_0$, respectively. The number of iterations equivalent to the original BP algorithm based on $\mathbf{H}$ is denoted as T.

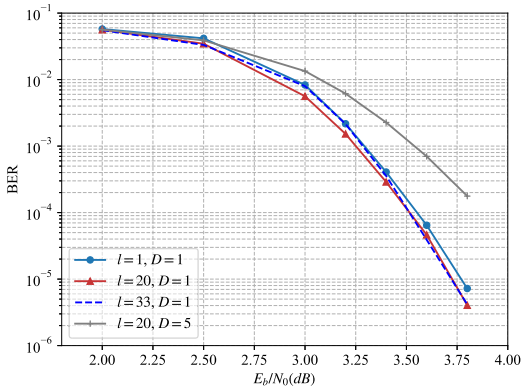


Fig. 6. Performance comparison of the NBP-RID decoder with different parameters.

As illustrated in Fig. 5, when $T = 5$, both NBP-RID and RID decoders outperform the SPA in higher signal-to-noise ratio (SNR) range and gradually approach the performance of the SPA with 100 iterations. Additionally, compared to the RID decoder, the NBP-RID decoder demonstrates significantly better performance when $l = 1$. When increasing the number of rows in $\mathbf{H}_0$, the performance of the NBP-RID at $l = 20$ remains superior to that of the RID at $l = 151$. In this case, the number of CN messages that need to be updated per iteration in NBP-RID is reduced by a factor of 7.55.

To illustrate the impact of different values of l and D on the NBP-RID, Fig. 6 presents a comparison of the BER for various l and D settings in the NBP-RID network. On one hand, increasing the grouping size l can enhance the performance of the NBP-RID decoder, though the improvement is small. The greater the grouping size is, the more CNs are involved in a sub-iteration. For this code, the critical condition derived from equation (13) is $1057/33 = 32.03$. In our experiments, no significant performance gains were observed for $l > 20$, which aligns with the selection strategy described in Section III-B.3. Based on our theoretical analysis and experimental results, $l = 1$ is recommended as it already achieves a good trade-off between performance and complexity. On the other hand, if we increase the depth D of the neural network beyond 1, the performance of NBP-RID decoder begins to decline noticeably. This happens because D determines the number of consecutive sub-iterations based on the same submatrix, which involves a group of the corresponding VNs and CNs within an *incomplete* Tanner graph. In such an incomplete Tanner graph, a CN or VN cannot receive complete messages from all its neighboring nodes. Therefore, compared to message propagation in a complete Tanner graph, the (NBP-) RID method is more prone to errors during message exchanges, leading to performance degradation. Multiple consecutive updates on the same set of VNs and CNs can further exacerbate these errors. Consequently, for the (NBP-) RID decoder, updating each set of CNs and VNs only once ($D = 1$) is recommended, as it helps mitigate the negative

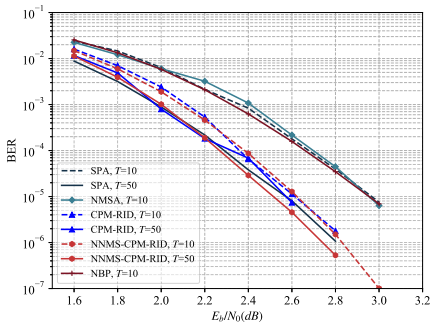


Fig. 7. BER results for (1040, 520) algebraic QC-LDPC code.

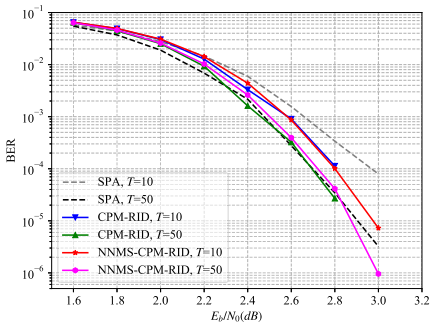


Fig. 8. BER results for (1560, 1040) algebraic QC-LDPC code.

effects of using an incomplete Tanner graph by incorporating messages from diverse sources (nodes) to some extent.

B. Performance of Revolving NNMS Decoders for QC-LDPC Codes

1) Algebraic QC-LDPC Codes: We consider the (1040, 520) QC-LDPC code with lifting size $Z = 130$ constructed via the algebraic approach in [5]. This code was originally one of the scenarios used to test the CPM-RID algorithm. The results for this code are presented in Fig. 7, where our proposed decoder is denoted as NNMS-CPM-RID. Note that T denotes the number of iterations for the SPA, MSA, and NBP decoder, while T denotes the equivalent number of complete iterations for the CPM-RID and NNMS-CPM-RID decoder. When $T = 10$, we trained our NNMS-CPM-RID decoder and a vanilla NBP decoder from scratch for comparison. When $T = 50$, we used the NNMS-CPM-RID decoder trained for $T = 10$ directly without any fine-tuning due to the weight-sharing across sub-iterations, and we failed to train a satisfactory vanilla NBP decoder due to hardware limitations, which also highlights the superiority of the proposed decoder in terms of GPU deployment difficulty.

As can be seen in Fig. 7, with reduced computational complexity and hardware implementation challenges, the CPM-RID decoder has already demonstrated sufficiently good performance for algebraic QC-LDPC codes. Our proposed decoder achieves only a small improvement compared to the CPM-RID decoder. Besides, at high SNRs, the bit error rate is slightly lower than that of the SPA decoder when $T = 50$.

Additionally, when $T = 10$ the bit error rates (BERs) of CPM-RID and NNMS-CPM-RID decoders are at a relatively lower level than other decoders, and when $T = 50$ the SPA, CPM-RID, and NNMS-CPM-RID decoders achieve similar performance. Therefore, the convergence rate of CPM-RID and NNMS-CPM-RID is faster than the SPA decoder. This phenomenon can be explained by the fact that CPM-RID and NNMS-CPM-RID decoders employ a layered decoding approach [32], [33], which allows for more timely message exchanges, leading to relatively better performance with fewer iterations. Although the traditional layered decoding also utilizes a subset of rows from the parity-check matrix during message updates, it is based on a complete Tanner graph. However, our method simplifies the Tanner graph during message updates, reducing the number of computations required for message updates.

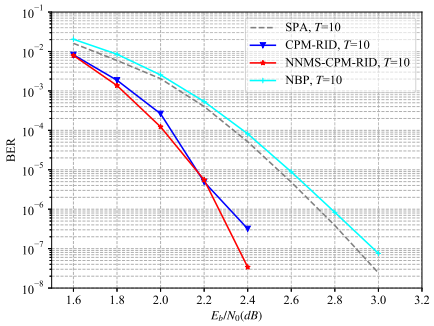


Fig. 9. BER results for (2640, 1320) algebraic QC-LDPC code.

Moreover, we present the decoding performance for (1560, 1040) and (2560, 1320) QC-LDPC codes obtained in an algebraic method [2], [6] in Fig. 8 and Fig. 9. Overall, for algebraic QC-LDPC codes, the proposed decoder demonstrates a slight advantage over the CPM-RID decoder at certain SNR points. Also, as T increases, the performance improvement is less pronounced compared to SPA. However, to achieve comparable performance, the SPA requires a greater number of equivalent iterations than (NBP-) RID, thereby increasing the computational complexity. Besides, as shown in Fig. 9, the performance of NBP (under the best conditions observed during our experiment) is actually worse than SPA, which is contrary to our expectation. This can be explained by the fact that as the block length increases, the number of parameters in the neural network also increases dramatically. Without good structural priors as constraints or effective training strategies [21], [30], it becomes challenging to obtain a satisfactory model.

2) 5G LDPC Codes: In order to evaluate the generalization of our proposed decoder, we consider protograph LDPC codes defined in the 5G standard [34]. We used the neural network that updates CN messages as (14) for an enhanced capability, denoted as NNOMS-CPM-RID decoder. The employed base code is BG2, the first two columns of which are always punctured. The lifting size is $Z = 16$ and $Z = 30$, thus an (800, 160) and a (1500, 300) 5G LDPC code is obtained according to the standard, respectively. These codes are also used in the literature [12].

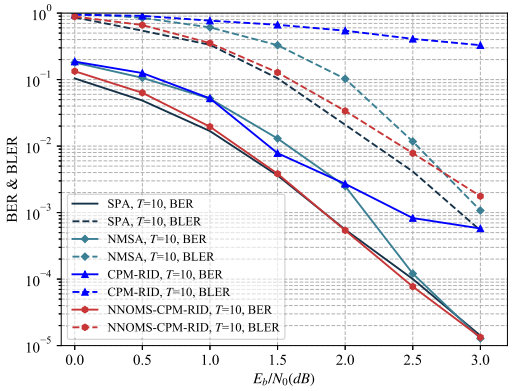


Fig. 10. BER and BLER results for (800, 160) 5G LDPC code.

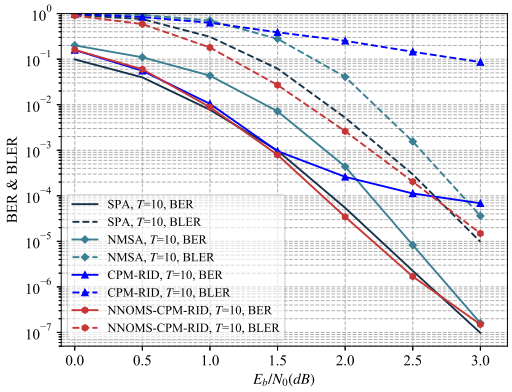


Fig. 11. BER and BLER results for (1500, 300) 5G LDPC code.

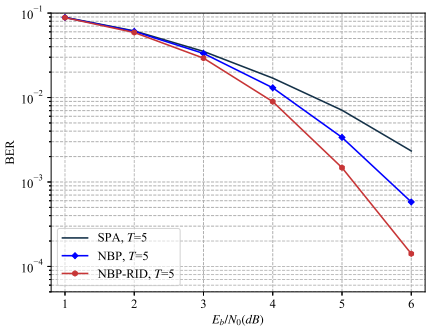


Fig. 12. BER results for (63, 45) BCH code.

The BER and BLER results for 5G LDPC code are presented in Fig. 10 and Fig. 11. Compared to the CPM-RID decoder, our NNOMS-CPM-RID decoder achieves a surprisingly significant improvement. Moreover, the NNOMS-CPM-RID decoder outperforms the NMSA decoder at lower SNRs, approaching the performance of the SPA decoder. This is achieved under the premise of remarkably reducing computational complexity and GPU memory consumption, as analyzed in Section IV-C.2

C. Other Results and Discussion

1) Decoding BCH Codes: Although the NBP-RID decoder proposed in Section III is primarily designed for decoding cyclic LDPC codes, it is essentially a simplified NBP decoder tailored for cyclic codes. Therefore, it can also be directly applied to cyclic HDPC codes, such as BCH codes. Fig. 12 displays the performance of NBP-RID decoder for the BCH (63,45) code, with parameter $l = 1$ and $D = 1$. In Fig. 12, the performance of the NBP-RID decoder surpasses that of both SPA and NBP decoders. It should be noted that among these decoders, only the NBP-RID decoder uses a simplified parity-check matrix (1×63) to build the network, while SPA and NBP decoders use the 18×63 parity-check matrix for decoding. Therefore, the proposed decoder still exhibits good performance for cyclic HDPC codes, indicating its mechanism's general applicability to cyclic codes.

Moreover, some cyclic codes can be applied in rate-compatible scenarios to better meet the demands of modern wireless communication systems [25], [35], or can be applied in distributed storage systems [36], [37]. Some of these codes, strictly speaking, are not cyclic codes but they exhibit structures that are highly similar to cyclic codes, making it feasible to extend the proposed decoder to such codes. Designing NBP-like decoders that support other cyclic code families is an attractive direction for future research.

2) Decoding Under the Rayleigh Channel: Robustness characterizes the performance of a neural network trained on a dataset with a specific distribution when tested on a dataset with a different distribution. To show the robustness of the proposed decoder, we test the NNOMS-CPM-RID decoder for the (800, 160) 5G LDPC code under the Rayleigh channel, and present the BER and BLER results in Fig. 13. The neural decoder trained under the AWGN channel in Fig. 10 is *directly* used in this figure. Compared to the BER curve in Fig. 10, which is only close to the performance of SPA, the proposed decoder demonstrates better performance than SPA under mismatched channel conditions. Additionally, the BLER curve of the proposed decoder is close to that of the SPA's BLER curve. This validates the robustness of the proposed method, making it a strong contender for forward error correction under Gaussian and non-Gaussian channel conditions, as it can achieve the goals as follows:

- Compared to NBP-like decoders in previous work: it is easier to train and achieve satisfactory results, and it requires fewer hardware resources for deployment.
- Compared to SPA, NMSA, and OMSA: it reduces the number of messages that need to be processed in each decoding iteration, thereby decreasing the overall computational complexity while maintaining competitive error correction performance.
- Compared to CPM-RID: it is applicable to a wider range of QC-LDPC codes and non-Gaussian channels.

3) Impact of Quantization: In Fig. 14, we show the impact on the performance of quantizing channel outputs, all weights, and messages between layers. Two common approaches are considered: post-training quantization (PTQ) and quantization-aware training (QAT) [38]. PTQ quantizes a fully trained floating-point model, whereas QAT integrates quantization operations during training. We mainly compare the 8-bit NNOMS-CPM-RID models with the 8-bit NMSA, and all decoders adopt uniform quantization. Compared to their floating-point counterparts, all quantized decoders exhibit

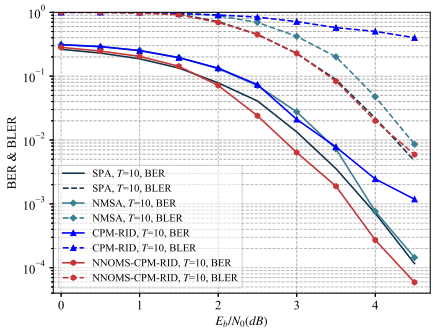


Fig. 13. BER and BLER results for (800, 160) 5G LDPC code under the Rayleigh channel.

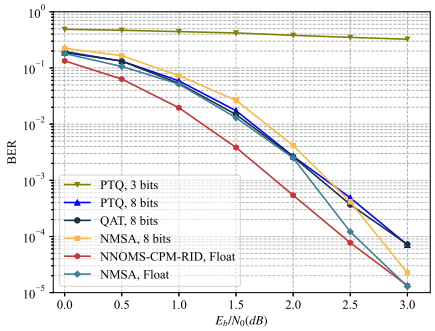


Fig. 14. Impact of quantization on performance for (800,160) 5G LDPC code.

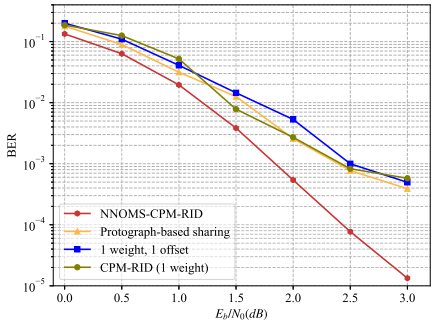


Fig. 15. BER results for 5G LDPC code with different parameter sharing strategies. The photograph-based sharing method is from [12].

performance degradation. For 8-bit quantization, PTQ outperforms NMSA in the lower SNR region but performs worse in the higher SNR region. QAT offers a slight improvement over PTQ, as it enables the model to adapt to quantization effects during training. During QAT, the straight-through estimator (STE) [39] is employed to enable gradient backpropagation through quantization operations. We also report 3-bit PTQ results, which show a noticeable performance degradation. Exploring how to better adapt neural decoder parameter training to quantization scenarios and how to select appropriate quantization methods will be interesting directions for future research.

4) About the Number of Parameters: As an additional reference, we here discuss the selection of the number of learnable (different) parameters in the model. Reducing parameter count with minimal performance loss is a reasonable trade-off. Therefore, our design balances training complexity, decoding performance, and iteration flexibility. We observed that an extreme reduction in the number of parameters, such as using only one weight and one offset per iteration, leads to great performance degradation, as is shown in Fig. 15. For comparison, the BER results obtained using the method in [12] are also provided. Although using independent weights and offsets for the messages on each iteration and each edge is most likely to yield the best performance, this approach tightly binds the parameters to a specific number of iterations. In contrast, our proposed decoder uses different weights and offsets for messages transmitted on different edges but shares them across iterations, thereby decoupling the parameters from their strong dependence on the iteration count. This allows for a more flexible choice of iteration count: when we apply the early stopping strategy used in traditional iterative message-passing algorithms to our proposed decoder, we can still achieve decoding performance close to that obtained with a fixed number of iterations.

VI. Conclusion

In this paper, we proposed a revolving framework for NBP-like decoders for cyclic and QC-LDPC codes, for lower complexity from the perspective of computation and hardware deployment. By extracting a special subset of rows from the original complete parity-check matrix and building a simplified neural decoder, the decoding process can be executed efficiently owing to the inherent structure of cyclic or QC-LDPC codes. Complexity analysis shows how an NBP-like decoder processes data in a matrix form, and the proposed scheme effectively reduces memory overhead. Compared to the previous decoding schemes, the proposed decoder can be applied to the algebraic QC-LDPC codes, and other classes of QC-LDPC codes such as 5G LDPC codes. Moreover, it outperforms the MSA and approaches the SPA at some SNRs. Additionally, the proposed decoder for cyclic LDPC codes can be applied to other classes of cyclic codes, which are important code families. By decoding under a mismatched channel, the proposed decoder also demonstrates its robustness. The proposed decoder extends the applicability of NBP-like decoders to longer codes, presenting broader possibilities in practice. For future work, exploring more QC-LDPC code classes and designing neural decoders for other classes of error-correcting codes is an intriguing topic. Finally, designing better training strategies for neural decoders of long codes to achieve efficient training is also a worthy area of focus and research.

References

[1] W. Ryan and S. Lin, *Channel Codes: Classical and Modern*. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[2] J. Li, K. Liu, S. Lin, and K. Abdel-Ghaffar, "Algebraic quasi-cyclic LDPC codes: Construction, low error-floor, large girth and a reduced-complexity decoding scheme," *IEEE Trans. Commun.*, vol. 62, no. 8, pp. 2626–2637, Aug. 2014.

[3] T.-Y. Chen, K. Vakilinia, D. Divsalar, and R. D. Wesel, "Protograph-based raptor-like LDPC codes," *IEEE Trans. Commun.*, vol. 63, no. 5, pp. 1522–1532, May 2015.

[4] K. Liu, S. Lin, and K. Abdel-Ghaffar, "A revolving iterative algorithm for decoding algebraic cyclic and quasi-cyclic LDPC codes," *IEEE Trans. Commun.*, vol. 61, no. 12, pp. 4816–4827, Dec. 2013.

[5] J. Li, K. Liu, S. Lin, and K. Abdel-Ghaffar, "Decoding of quasi-cyclic LDPC codes with section-wise cyclic structure," in *Proc. Inf. Theory Appl. Workshop (ITA)*, Feb. 2014, pp. 1–10.
[6] S. Lin, K. Liu, J. Li, and K. Abdel-Ghaffar, "A reduced-complexity iterative scheme for decoding quasi-cyclic low-density parity-check codes," in *Proc. 48th Asilomar Conf. Signals, Syst. Comput.*, Nov. 2014, pp. 119–125.
[7] E. Nachmani, Y. Be'ery, and D. Burshtein, "Learning to decode linear codes using deep learning," in *Proc. 54th Annu. Allerton Conf. Commun., Control, Comput. (Allerton)*, Sep. 2016, pp. 341–346.
[8] E. Nachmani, E. Marciano, L. Lugosch, W. J. Gross, D. Burshtein, and Y. Be'ery, "Deep learning methods for improved decoding of linear codes," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 119–131, Feb. 2018.
[9] L. Lugosch and W. J. Gross, "Neural offset min-sum decoding," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2017, pp. 1361–1365.
[10] A. Buchberger, C. Häger, H. D. Pfister, L. Schmalen, and A. Graell i Amat, "Pruning and quantizing neural belief propagation decoders," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 1957–1966, Jul. 2021.
[11] X. Chen and M. Ye, "Cyclically equivariant neural decoders for cyclic codes," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2021, pp. 1771–1780.
[12] J. Dai et al., "Learning to decode protograph LDPC codes," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 1983–1999, Jul. 2021.
[13] E. Nachmani and Y. Be'ery, "Neural decoding with optimization of node activations," *IEEE Commun. Lett.*, vol. 26, no. 11, pp. 2527–2531, Nov. 2022.
[14] N. Shah and Y. Vasavada, "Neural layered decoding of 5G LDPC codes," *IEEE Commun. Lett.*, vol. 25, no. 11, pp. 3590–3593, Nov. 2021.
[15] L. Wang, S. Chen, J. Nguyen, D. Dariush, and R. Wesel, "Neural-network-optimized degree-specific weights for LDPC MinSum decoding," in *Proc. 11th Int. Symp. Topics Coding (ISTC)*, Aug. 2021, pp. 1–5.
[16] Y. Tang, L. Zhou, S. Zhang, C. Chen, and L. Wang, "Normalized neural network for belief propagation LDPC decoding," in *Proc. IEEE Int. Conf. Netw., Sens. Control (ICNSC)*, vol. 1, Dec. 2021, pp. 1–5.
[17] Q. Wang, S. Wang, H. Fang, L. Chen, L. Chen, and Y. Guo, "A model-driven deep learning method for normalized min-sum LDPC decoding," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, Jun. 2020, pp. 1–6.
[18] A. Bennatan, Y. Choukroun, and P. Kisilev, "Deep learning for decoding of linear codes—A syndrome-based approach," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2018, pp. 1595–1599.
[19] Y. Choukroun and L. Wolf, "Error correction code transformer," in *Proc. Adv. Neural Inf. Process. Syst. 35: Annu. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2022, pp. 38695–38705.
[20] Y. Choukroun and L. Wolf, "Denoising diffusion error correction codes," in *Proc. 11th Int. Conf. Learn. Represent. (ICLR)*, 2022, pp. 4053–4070.
[21] H.-Y. Kwak, D.-Y. Yun, Y. Kim, S. Kim, and J. No, "Boosting learning for LDPC codes to improve the error-floor performance," in *Proc. Adv. Neural Inf. Process. Syst. 36, Annu. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2023, pp. 22115–22131.
[22] Q. Zhang, B. Chen, T. Zhuang, Y. Jiang, and S.-T. Xia, "Section-wise revolving NBP-like decoders for QC-LDPC codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jul. 2024, pp. 1397–1402.
[23] S. Landner and O. Milenkovic, "Algorithmic and combinatorial analysis of trapping sets in structured LDPC codes," in *Proc. Int. Conf. Wireless Netw., Commun. Mobile Comput.*, Jun. 2005, pp. 630–635.
[24] Q. Huang, Q. Diao, S. Lin, and K. Abdel-Ghaffar, "Cyclic and quasi-cyclic LDPC codes on constrained parity-check matrices and their trapping sets," *IEEE Trans. Inf. Theory*, vol. 58, no. 5, pp. 2648–2671, May 2012.
[25] M. Battaglioni, M. Baldi, F. Chiaraluce, and G. Cancellieri, "Rate-compatible LDPC codes based on primitive polynomials and Golomb rulers," *IEEE Trans. Commun.*, vol. 72, no. 12, pp. 7361–7373, Dec. 2024.
[26] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NIPS*, Dec. 2019, pp. 8024–8035.
[27] M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of Machine Learning*. Cambridge, MA, USA: MIT Press, 2018.
[28] S. Adiga, X. Xiao, R. Tandon, B. Vasić, and T. Bose, "Generalization bounds for neural belief propagation decoders," *IEEE Trans. Inf. Theory*, vol. 70, no. 6, pp. 4280–4296, Jun. 2024.
[29] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[30] H.-Y. Kwak, D.-Y. Yun, Y. Kim, S.-H. Kim, and J.-S. No, "Boosted neural decoders: Achieving extreme reliability of LDPC codes for 6G networks," 2024, *arXiv:2405.13413*.
[31] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, 2014, pp. 1–15.
[32] N. Jiang, K. Peng, J. Song, C. Pan, and Z. Yang, "High-throughput QC-LDPC decoders," *IEEE Trans. Broadcast.*, vol. 55, no. 2, pp. 251–259, Jun. 2009.
[33] K. Zhang, X. Huang, and Z. Wang, "High-throughput layered decoder implementation for quasi-cyclic LDPC codes," *IEEE J. Sel. Areas Commun.*, vol. 27, no. 6, pp. 985–994, Aug. 2009.
[34] *5G; NR; Multiplexing and Channel Coding*, document TS 38.212, 3GPP, 2022.
[35] M. Shirvanimoghaddam, "Primitive rateless codes," *IEEE Trans. Commun.*, vol. 69, no. 10, pp. 6395–6408, Oct. 2021.
[36] B. Chen, W. Fang, S.-T. Xia, and F.-W. Fu, "Constructions of optimal (r, δ) locally repairable codes via constacyclic codes," *IEEE Trans. Commun.*, vol. 67, no. 8, pp. 5253–5263, Aug. 2019.
[37] B. Chen, S.-T. Xia, J. Hao, and F.-W. Fu, "Constructions of optimal cyclic (r, δ) locally repairable codes," *IEEE Trans. Inf. Theory*, vol. 64, no. 4, pp. 2499–2511, Apr. 2018.
[38] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Micikevicius, "Integer quantization for deep learning inference: Principles and empirical evaluation," 2020, *arXiv:2004.09602*.
[39] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.

Qinshan Zhang received the B.E. degree in electronic information engineering from Beihang University (BUAA), China, in 2021, and the M.E. degree in computer technology from Tsinghua University, China, in 2024. He is currently pursuing the joint Ph.D. degree with Tsinghua Shenzhen International Graduate School, Tsinghua University, China, and the Peng Cheng Laboratory, Shenzhen, China. His research interests include channel coding and machine learning.

Bin Chen (Member, IEEE) received the B.S. and M.S. degrees in mathematics from South China Normal University, Guangzhou, China, in 2014 and 2017, respectively, and the Ph.D. degree in computer science and technology from the Department of Computer Science and Technology, Tsinghua University, Beijing, China, in 2021. From December 2019 to May 2020, he visited the Department of Electrical and Computer Engineering, University of Waterloo, Waterloo, ON, Canada. From May 2021 to November 2021, he was a Post-Doctoral Researcher with Shenzhen International Graduate School, Tsinghua University, Shenzhen, China. Since December 2021, he has been an Associate Professor with the Department of Computer Science and Technology, Harbin Institute of Technology (Shenzhen). He has published more than 50 technical papers at prominent journals and conferences, such as IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, *International Journal of Computer Vision*, ICML, NeurIPS, ICLR, CVPR, ICCV, ECCV, AAAI, IJCAI, and ACM MM. His research interests include coding and information theory, multimedia compression, and AI safety. He served as the Area Chair for NeurIPS and ICLR.

Tianqu Zhuang received the B.E. degree in computer science and the B.S. degree in mathematics from Sun Yat-sen University (SYSU), China, in 2023. He is currently pursuing the M.E. degree in computer science with Tsinghua Shenzhen International Graduate School, Tsinghua University, China. His research interests include deep learning, information theory, and recommendation systems.

Yong Jiang (Member, IEEE) received the B.S. and Ph.D. degrees in computer science and technology from Tsinghua University, Beijing, China. Currently, he is a Full Professor with Tsinghua Shenzhen International Graduate School. His research interests include future network architecture, the internet QoS, and network function virtualization.

Shu-Tao Xia (Member, IEEE) received the B.S. degree in mathematics and the Ph.D. degree in applied mathematics from Nankai University, Tianjin, China, in 1992 and 1997, respectively. From March 1997 to April 1999, he was with the Research Group of Information Theory, Department of Mathematics, Nankai University. Since January 2004, he has been with Tsinghua Shenzhen International Graduate School, Guangdong, China, where he is currently a Full Professor. His current research interests include coding and information theory, networking, and machine learning.